

Defense guide: eval-churn-gate

For reading before an interview. Every number here is re-measured by `tools/collect_metrics.py` and checked against this file by `tools/check_numbers.py`, so a stale sentence fails the build rather than getting read aloud.

The 30 second version

"A regression gate on an aggregate score is a function of two numbers, and two numbers cannot tell you whether the same items improved or a different set improved and regressed in equal measure. I built a corpus where the true effect is known, including comparisons of a version against itself where it is exactly zero, and measured what each gate does. The gate almost everyone ships catches 0.323 of the regressions and misses both of the ones whose aggregate is about zero."

Then stop.

The question that usually comes next is "what would you ship instead", and the useful part of the answer is that it depends on which mistake costs more, and I measured both sides.

The claims, and how each one is proved

Claim 1: three of five gates cannot see a compensating regression

Command: `python -m churninggate gate --scenario compensating`

One slice loses 0.1173 while the aggregate moves by 0.0006 in the median case. The mean threshold gate sees it 0.0 of the time. The paired tests fire at 0.022 against their own false block rate of about 0.04, so they are not detecting it, they are firing at their own noise rate. The slice scan catches it 1.0 of the time.

If pushed on "so the slice scan is the answer": it misses pure churn, catching only 0.028 of it, while the churn gate catches that one 1.0. Two blind spots, two gates, and the combined gate is what covers both.

Claim 2: the impossibility has a number attached

Command: `python experiments/exp04_the_aggregate_carries_no_churn_information.py`

Across 2133 comparisons whose delta is within five thousandths of zero, the share of items that changed side runs from 0.0667 to 0.5933 of items, with a rank correlation of 0.001 in magnitude and a p value of 0.963. Restricted to the 2000 comparisons whose true aggregate effect is exactly zero, the correlation there is 0.0027 in magnitude.

If pushed on "correlation is not the same as no information": correct, and that is why the structural argument comes first. The aggregate is a function of the difference between two transition counts and the churn is a function of their sum, so the independence is a property of the arithmetic. The correlation is the measurement that confirms it rather than the reason to believe it.

Claim 3: pairing is worth a factor of four and costs nothing

Command: `python -m churninggate power`

At the shipped 300 item eval set the smallest detectable regression is 0.0122 paired, against 0.0463 when two averages are compared. That is worth a factor of 3.8 on the same data. Detecting the configured threshold needs 112 items paired, or 1608 comparing two averages.

If pushed on "why does anyone compare two averages then": because the per item scores were averaged away before anything was written down. The cost of pairing is keeping them, which most pipelines already do in memory.

Claim 4: a threshold set at the regression size is a coin flip

Command: `python experiments/exp03_what_the_eval_set_can_resolve.py`

At a true drop equal to the configured threshold, the mean threshold gate catches it 0.455 of the time it happens, against 0.99 for the paired interval on the same data. The observed value lands either side of the threshold with roughly equal probability, so a gate set at the size of the thing it was built for is a coin flip on it.

If pushed on "then just lower the threshold": that raises the false block rate, and the point of the table is that the paired interval gets the detection without the trade because it uses the information the aggregate threw away.

Claim 5: the gate publishes its own error rates

Command: `python experiments/exp01_the_gate_graded_against_nothing.py`

Over 1000 comparisons whose true effect is exactly zero, the combined gate catches 1.0, at a false block rate of 0.037 across the clean scenarios. The paired interval alone blocks 0.039 of clean A/A comparisons, and the extra conditions are what buy the two scenarios it misses entirely. The churn gate catches 0.384 overall while blocking only 0.005.

If pushed on "your A/A comparisons are synthetic": they are, and that is what makes the zero exact rather than approximate. A real A/A pair is only A/A if nothing else changed, which is the thing nobody can verify.

Questions that are meant to be hard

Is this just a paired t test? The paired test is one of five gates and it is one of the three that cannot see a compensating regression. What is mine: the transition decomposition with the net against gross split, the A/A anchor and the three quantities it makes measurable, the slice scan with a Holm correction and the measurement of what it costs, the third verdict, and the receipts pipeline that fails the build when a document quotes a number the code no longer produces. Around eight hundred statements of source, 206 tests, 99.3 percent line coverage.

Your scores are generated. Does any of it transfer? Two kinds of claim. The structural one, that an aggregate cannot bound the churn, is arithmetic and transfers everywhere. The magnitudes are properties of these noise parameters, a run to run standard deviation of 0.06 and an item spread of 0.22, and the tool reads a policy file precisely so a team can put their own measured numbers in it.

What is the weakest part? The noise model has two components and reality has more. A real eval has run level effects that shift every item together, which pairing cannot remove and more items cannot shrink, and this model has no such term. So every minimum detectable effect here is the optimistic case. Second weakest: the slice scan only works because the losing group is attached to an observable label. A regression confined to an unobservable subgroup is invisible to every gate here.

What would you do differently with more time? Read a real eval log rather than generating one, because the per item scores are already there in most pipelines and that would turn every magnitude from illustrative into real. Then add the run level noise term. Then estimate the noise parameters from a team's own repeated runs instead of taking them from a policy file.

Why did you not use statsmodels? Because the power arithmetic is the most quoted thing in the repository and it is four lines, so importing it would mean the headline number comes from a function a reviewer cannot check in

thirty seconds. ADR-004 also says the honest version: if this were a production tool rather than a repository whose purpose is to show the reasoning, the trade would go the other way.

Did anything go wrong while you built it? Four things, and two are worth telling. The churn threshold was written from intuition at 0.06 and the measured A/A floor is about twice that, so the gate blocked every comparison whose true effect was exactly zero; the policy loader now refuses a limit below what the configured noise produces. And an early run measured a false block rate of 0.084 for a test whose true level is 0.05, at two hundred and fifty replications. It looked like a real finding for about an hour. At three thousand replications it was 0.055. That is why every rate in the repository now carries a Wilson interval and there is an explicit method for asking whether an interval excludes a value.

Why three verdicts rather than two? Because on comparisons whose true effect is exactly zero, the honest answer is "this eval set cannot resolve a difference this small" 0.928 of the time the honest answer is exactly that, and a two outcome gate returns pass for all of them. It blocked 0.069 of them and passed only 0.003. A pass is read as evidence of no regression, and there was no evidence either way.

Things to say, and things not to say

Say:

- "the aggregate is a function of the difference and the churn is a function of the sum, so one cannot bound the other."
- "the A/A comparison makes the false block rate a measurement rather than an assumption."
- "the combined gate costs 0.037 on clean releases, and here is what that buys."
- "the minimum detectable effect here is the optimistic case, because the noise model has no run level term."

Do not say:

- "zero false blocks." Say the mean threshold gate blocked none, which supports below 0.05 percent over 2000 clean releases and nothing stronger.
- "the paired t test runs hot." Its measured false block rate is not distinguishable from the nominal level at ninety five percent confidence, and an early run of this suite made exactly that mistake.
- "chunking the metric differently fixes it." A pass rate is the net of the transitions, so it is precisely the quantity that cancels compensating movement: 0.8889 of that movement is cancelled by the net on A/A comparisons.
- "99.3 percent coverage means it is correct." It means the lines ran. The A/A anchor is the thing that means something.

The live demo, five commands

```
# 1. What this eval set can resolve, before running any comparison.
python -m churninggate power

# 2. A slice collapses and the aggregate does not move. Exit 1.
python -m churninggate gate --scenario compensating

# 3. The gate almost everyone ships, on the same comparison. It passes.
python -m churninggate one --gate mean_threshold --scenario compensating

# 4. Every gate on every scenario, with an interval on every rate.
python -m churninggate sweep --replications 200

# 5. Every published figure re-measured and checked against the documents.
make receipts
```